

Particle Identifier and Decay Event Subsystem

VSEPR-SIM — Code Reference Draft

include/physics/ include/kinetics/ legacy/c_api/ tests/

v4.0.0 June 2025

Contents

1	Overview	2
2	particle_id.hpp — Identity Layer	2
2.1	The ParticleID Enum	2
2.2	Constexpr Query Functions	3
2.3	Query Classification Table	5
3	decay_event.hpp — Event Record	5
3.1	Field Semantics	6
4	decay_event_builder.hpp — Factory Layer	6
4.1	Nuclear State and Daughter Arithmetic	6
4.2	Builder Factories	7
4.3	Energy Partitioning Defaults	8
5	kinetic_event_kind.hpp — Classification	8
6	kinetic_event_record.hpp — Kinetics Wrapper	9
6.1	Dispatch Pattern	9
7	Plain C Legacy API	10
8	Test Suite — test_particle_id_decay.cpp	10
8.1	Test Results	12
9	Integration with Existing Infrastructure	12
9.1	Orthogonality with DecayMode and DecayType	12
9.2	Relationship to ElementDescriptor	12
9.3	Layer Mapping	13

Overview

The particle identifier subsystem defines a signed-integer identity namespace that spans ordinary matter species, emitted decay particles, radiation bookkeeping tokens, and defect physics sentinels. Every entity the engine emits, tracks, or accounts for is tagged with a value of type `ParticleID`.

$$ID_p \in \mathbb{Z}, \quad \begin{cases} ID_p \geq 0 & \text{ordinary matter (atoms, isotopes, molecules, candidates)} \\ ID_p < 0 & \text{emitted particles, energy stand-ins, defect tokens} \end{cases}$$

The entire scheme is constexpr-queryable at compile time. No runtime tables, no lookup files, no hidden state.

Header	Namespace	Contents
<code>include/physics/particle_id.hpp</code>	<code>vsepr::physics</code>	<code>ParticleID</code> enum + 5 constexpr queries
<code>include/physics/decay_event.hpp</code>	<code>vsepr::physics</code>	<code>DecayEvent</code> struct
<code>include/physics/decay_event_builder.hpp</code>	<code>vsepr::physics</code>	Builder factories + daughter arithmetic
<code>include/kinetics/kinetic_event_kind.hpp</code>	<code>vsepr::kinetics</code>	<code>EventKind</code> enum
<code>include/kinetics/kinetic_event_record.hpp</code>	<code>vsepr::kinetics</code>	<code>KineticEventRecord</code> + <code>EventPayload</code>
<code>legacy/c.api/particle_ids.h/.c</code>	— (plain C)	C enum + 4 query functions
<code>legacy/c.api/decay_event.h/.c</code>	— (plain C)	C struct + init/add

particle_id.hpp — Identity Layer

The ParticleID Enum

`ParticleID` is a scoped enum with `std::int32_t` backing. Negative values are reserved. Positive values are assigned at runtime by the species registry.

```
1 namespace vsepr::physics {
2
3 enum class ParticleID : std::int32_t {
4     nuclear_placeholder      = 0,
5
6     // Emitted decay particles
7     alpha_particle          = -1,
8     beta_minus               = -2,
9     gamma_photon             = -3,
10    energy_packet_generic     = -4,
11    beta_plus                 = -5,
12    electron_capture_token    = -6,
13    neutron_emission          = -7,
14    proton_emission           = -8,
15    neutrino_loss_token       = -9,
16
17    // Defect tokens
18    vacancy_token             = -10,
19    interstitial_token        = -11,
20    defect_cluster_token      = -12,
21    helium_bubble_seed        = -13,
22    lattice_damage_packet     = -14,
23 };
24
25 } // namespace vsepr::physics
```

Listing 1: `include/physics/particle_id.hpp` — enum definition

The enum partitions into three sub-ranges that the constexpr queries exploit directly:

$$\underbrace{[-14, -10]}_{\text{defect tokens}} \quad \underbrace{[-9, -1]}_{\text{emitted particles}} \quad \underbrace{\{0\}}_{\text{placeholder}} \quad \underbrace{[1, +\infty)}_{\text{matter species (runtime)}}$$

Constexpr Query Functions

All five queries are `[[nodiscard]] constexpr bool` noexcept. They cost zero runtime cycles when the argument is a compile-time constant.

```
1 // Reserved: 0 through -99 (the full engine sentinel range)
2 [[nodiscard]] constexpr bool is_reserved_particle_id(ParticleID id)
3     noexcept {
4     const auto v = static_cast<std::int32_t>(id);
5     return (v <= 0 && v >= -99);
6 }
7 // Transportable: physically propagates through matter
8 [[nodiscard]] constexpr bool is_transportable(ParticleID id) noexcept
9     {
10     switch (id) {
11     case ParticleID::alpha_particle:
12     case ParticleID::beta_minus:
13     case ParticleID::gamma_photon:
14     case ParticleID::beta_plus:
15     case ParticleID::neutron_emission:
16     case ParticleID::proton_emission:
17         return true;
18     default:
19         return false;
20     }
21 }
22 // Bookkeeping only: token exists for accounting, never transported
23 [[nodiscard]] constexpr bool is_bookkeeping_only(ParticleID id)
24     noexcept {
25     switch (id) {
26     case ParticleID::energy_packet_generic:
27     case ParticleID::electron_capture_token:
28     case ParticleID::neutrino_loss_token:
29         return true;
30     default:
31         return false;
32     }
33 }
34 // Defect token: range check [-14, -10]
35 [[nodiscard]] constexpr bool is_defect_token(ParticleID id) noexcept
36     {
37     const auto v = static_cast<std::int32_t>(id);
38     return (v <= -10 && v >= -14);
39 }
```

```

40 // String label for logging and output
41 [[nodiscard]] constexpr std::string_view to_string(ParticleID id)
    noexcept {
42     switch (id) {
43         case ParticleID::nuclear_placeholder:    return "
            nuclear_placeholder";
44         case ParticleID::alpha_particle:         return "
            alpha_particle";
45         case ParticleID::beta_minus:             return "beta_minus";
46         case ParticleID::gamma_photon:          return "gamma_photon
            ";
47         case ParticleID::energy_packet_generic:  return "
            energy_packet_generic";
48         case ParticleID::beta_plus:             return "beta_plus";
49         case ParticleID::electron_capture_token: return "
            electron_capture_token";
50         case ParticleID::neutron_emission:       return "
            neutron_emission";
51         case ParticleID::proton_emission:        return "
            proton_emission";
52         case ParticleID::neutrino_loss_token:    return "
            neutrino_loss_token";
53         case ParticleID::vacancy_token:         return "
            vacancy_token";
54         case ParticleID::interstitial_token:     return "
            interstitial_token";
55         case ParticleID::defect_cluster_token:   return "
            defect_cluster_token";
56         case ParticleID::helium_bubble_seed:    return "
            helium_bubble_seed";
57         case ParticleID::lattice_damage_packet:  return "
            lattice_damage_packet";
58         default:                                return "
            unknown_particle_id";
59     }
60 }

```

Listing 2: Constexpr queries — particle_id.hpp

Query Classification Table

ParticleID	reserved	transportable	bookkeeping	defect
nuclear_placeholder	✓			
alpha_particle	✓	✓		
beta_minus	✓	✓		
gamma_photon	✓	✓		
energy_packet_generic	✓		✓	
beta_plus	✓	✓		
electron_capture_token	✓		✓	
neutron_emission	✓	✓		
proton_emission	✓	✓		
neutrino_loss_token	✓		✓	
vacancy_token	✓			✓
interstitial_token	✓			✓
defect_cluster_token	✓			✓
helium_bubble_seed	✓			✓
lattice_damage_packet	✓			✓

decay_event.hpp — Event Record

DecayEvent is a plain aggregate that captures the full information of a single nuclear transition. Default member initialisation means a value-constructed DecayEvent{} is always zero-safe.

```

1 namespace vsepr::physics {
2
3 struct DecayEvent {
4     static constexpr std::size_t max_emitted_particles = 8;
5
6     std::int32_t parent_species_id {0};
7     std::int32_t daughter_species_id {0};
8
9     std::array<ParticleID, max_emitted_particles>
10         emitted_particle_ids {};
11     std::size_t emitted_count {0};
12
13     double released_energy_eV {0.0};
14     double deposited_energy_fraction {0.0};
15     double transported_energy_fraction {0.0};
16     double local_damage_score {0.0};
17
18     double event_time_s {0.0};
19     double confidence {0.0};
20
21     [[nodiscard]] bool add_emitted(ParticleID id) noexcept {
22         if (emitted_count >= emitted_particle_ids.size()) {
23             return false;
24         }
25         emitted_particle_ids[emitted_count++] = id;
26         return true;
27     }
28 };
29 } // namespace vsepr::physics

```

Field Semantics

Field	Type	Meaning
parent_species_id	int32_t	Engine-assigned ID of parent nuclide
daughter_species_id	int32_t	Engine-assigned ID of daughter nuclide
emitted_particle_ids	array<ParticleID,8>	Reserved IDs of emitted particles
emitted_count	size_t	How many slots are populated
released_energy_eV	double	Total Q-value in electron-volts
deposited_energy_fraction	double	Fraction absorbed locally $\in [0, 1]$
transported_energy_fraction	double	Fraction carried away $\in [0, 1]$
local_damage_score	double	Lattice displacement proxy $\in [0, 1]$
event_time_s	double	Engine time of event in seconds
confidence	double	Source reliability $\in [0, 1]$

The invariant `deposited_energy_fraction + transported_energy_fraction  $\approx$  1` is enforced by the builder factories but not by the struct itself, which remains a plain data holder.

decay_event_builder.hpp — Factory Layer

Nuclear State and Daughter Arithmetic

`NuclearState` is a minimal struct carrying only the fields needed for transition arithmetic. It does *not* replace `ElementDescriptor` (the full identity layer).

```

1 namespace vsepr::physics {
2
3 struct NuclearState {
4     std::int32_t species_id {0};
5     int Z {0};
6     int A {0};
7 };
8
9 struct DaughterState {
10     int Z {0};
11     int A {0};
12 };
13
14 [[nodiscard]] constexpr DaughterState
15 alpha_daughter(const NuclearState& s) noexcept {
16     return DaughterState{ s.Z - 2, s.A - 4 };
17 }
18
19 [[nodiscard]] constexpr DaughterState
20 beta_minus_daughter(const NuclearState& s) noexcept {
21     return DaughterState{ s.Z + 1, s.A };
22 }
23
24 [[nodiscard]] constexpr DaughterState
25 beta_plus_daughter(const NuclearState& s) noexcept {
26     return DaughterState{ s.Z - 1, s.A };
27 }

```

```

28
29 } // namespace vsepr::physics

```

Listing 4: Transition structs and constexpr daughter arithmetic

The arithmetic corresponds directly to the standard nuclear transition equations:

$$\begin{aligned}
\alpha &: S(Z, A) \rightarrow S(Z - 2, A - 4) + P_{-1} \\
\beta^- &: S(Z, A) \rightarrow S(Z + 1, A) + P_{-2} + P_{-9} \\
\beta^+ &: S(Z, A) \rightarrow S(Z - 1, A) + P_{-5} + P_{-9}
\end{aligned}$$

Builder Factories

Each factory populates a `DecayEvent` with mode-specific energy partitioning defaults and the correct emitted-particle list.

```

1 // Alpha: heavy, short-range, high local damage
2 // Deposited 0.85 / Transported 0.15 / Damage 0.90
3 [[nodiscard]] inline DecayEvent make_alpha_decay(
4     std::int32_t parent_species_id,
5     std::int32_t daughter_species_id,
6     double released_energy_eV,
7     double event_time_s,
8     double confidence
9 ) {
10     DecayEvent ev {};
11     ev.parent_species_id = parent_species_id;
12     ev.daughter_species_id = daughter_species_id;
13     ev.released_energy_eV = released_energy_eV;
14     ev.event_time_s = event_time_s;
15     ev.confidence = confidence;
16     ev.deposited_energy_fraction = 0.85;
17     ev.transported_energy_fraction = 0.15;
18     ev.local_damage_score = 0.90;
19     (void)ev.add_emitted(ParticleID::alpha_particle);
20     return ev;
21 }

```

Listing 5: Alpha decay factory

```

1 // Beta-minus: light, medium range, moderate damage
2 // Deposited 0.35 / Transported 0.65 / Damage 0.30
3 [[nodiscard]] inline DecayEvent make_beta_minus_decay(
4     std::int32_t parent_species_id,
5     std::int32_t daughter_species_id,
6     double released_energy_eV,
7     double event_time_s,
8     double confidence
9 ) {
10     DecayEvent ev {};
11     ev.parent_species_id = parent_species_id;
12     ev.daughter_species_id = daughter_species_id;
13     ev.released_energy_eV = released_energy_eV;
14     ev.event_time_s = event_time_s;
15     ev.confidence = confidence;
16     ev.deposited_energy_fraction = 0.35;
17     ev.transported_energy_fraction = 0.65;

```

```

18     ev.local_damage_score      = 0.30;
19     (void)ev.add_emitted(ParticleID::beta_minus);
20     (void)ev.add_emitted(ParticleID::neutrino_loss_token);
21     return ev;
22 }

```

Listing 6: Beta-minus decay factory

```

1  // Gamma: penetrating, low local damage, high transport
2  // Deposited 0.10 / Transported 0.90 / Damage 0.05
3  [[nodiscard]] inline DecayEvent make_gamma_decay(
4      std::int32_t parent_species_id,
5      std::int32_t daughter_species_id,
6      double released_energy_eV,
7      double event_time_s,
8      double confidence
9  ) {
10     DecayEvent ev {};
11     ev.parent_species_id    = parent_species_id;
12     ev.daughter_species_id  = daughter_species_id;
13     ev.released_energy_eV   = released_energy_eV;
14     ev.event_time_s         = event_time_s;
15     ev.confidence           = confidence;
16     ev.deposited_energy_fraction = 0.10;
17     ev.transported_energy_fraction = 0.90;
18     ev.local_damage_score   = 0.05;
19     (void)ev.add_emitted(ParticleID::gamma_photon);
20     return ev;
21 }

```

Listing 7: Gamma decay factory

Energy Partitioning Defaults

Factory	Deposited	Transported	Damage score
make_alpha_decay	0.85	0.15	0.90
make_beta_minus_decay	0.35	0.65	0.30
make_gamma_decay	0.10	0.90	0.05

kinetic_event_kind.hpp — Classification

EventKind is the coarse universal classifier that sits at the top of the kinetics layer. Formation-specific EventType values from formation_kinetics.hpp (28-type taxonomy) map into the 11 kinds here at the scheduling layer.

```

1  namespace vsepr::kinetics {
2
3  enum class EventKind : std::int32_t {
4      unknown          = 0,
5      decay,
6      transport,
7      collision,
8      adsorption,
9      desorption,
10     coalescence,
11     nucleation,

```



```

12     deposition,
13     restructuring,
14     phase_change
15 };
16
17 } // namespace vsepr::kinetics

```

Listing 8: include/kinetics/kinetic_event_kind.hpp

kinetic_event_record.hpp — Kinetics Wrapper

KineticEventRecord wraps any physics-specific event in a single schedulable unit. The payload is a `std::variant` so the kinetic scheduler never needs to cast or union-punch.

```

1  namespace vsepr::kinetics {
2
3  struct GenericTransportEvent {
4      int source_id {0};
5      int target_id {0};
6      double time_s {0.0};
7      double confidence {0.0};
8  };
9
10 using EventPayload = std::variant<
11     vsepr::physics::DecayEvent,
12     GenericTransportEvent
13 >;
14
15 struct KineticEventRecord {
16     EventKind kind {EventKind::unknown};
17     double intensity {0.0};
18     double barrier_score {0.0};
19     double environment_multiplier {1.0};
20     EventPayload payload {};
21 };
22
23 } // namespace vsepr::kinetics

```

Listing 9: include/kinetics/kinetic_event_record.hpp

Dispatch Pattern

The canonical dispatch pattern over EventPayload uses `std::get_if`, which is zero-overhead on most ABIs:

```

1  void dispatch(const KineticEventRecord& rec) {
2      if (auto* d = std::get_if<vsepr::physics::DecayEvent>(&rec.
3          payload)) {
4          // nuclear decay path
5          process_decay(*d);
6      } else if (auto* t = std::get_if<GenericTransportEvent>(&rec.
7          payload)) {
8          // transport path
9          process_transport(*t);
10     }
11 }

```

Listing 10: Variant dispatch at the scheduling layer

Plain C Legacy API

The plain C bridge in `legacy/c_api/` mirrors the C++ enum with `SCREAMING_SNAKE_CASE` constants and provides four query functions via `extern "C"` linkage.

```
1 typedef enum ParticleID {
2     PARTICLE_NUCLEAR_PLACEHOLDER      = 0,
3
4     PARTICLE_ALPHA                     = -1,
5     PARTICLE_BETA_MINUS                 = -2,
6     PARTICLE_GAMMA                     = -3,
7     PARTICLE_ENERGY_PACKET              = -4,
8     PARTICLE_BETA_PLUS                  = -5,
9     PARTICLE_ELECTRON_CAPTURE           = -6,
10    PARTICLE_NEUTRON_EMISSION            = -7,
11    PARTICLE_PROTON_EMISSION              = -8,
12    PARTICLE_NEUTRINO_LOSS               = -9,
13
14    PARTICLE_VACANCY                     = -10,
15    PARTICLE_INTERSTITIAL                 = -11,
16    PARTICLE_DEFECT_CLUSTER               = -12,
17    PARTICLE_HELIUM_BUBBLE_SEED          = -13,
18    PARTICLE_LATTICE_DAMAGE_PACKET        = -14
19 } ParticleID;
20
21 const char* particle_id_name(int id);
22 int particle_id_is_reserved(int id);
23 int particle_id_is_transportable(int id);
24 int particle_id_is_bookkeeping_only(int id);
```

Listing 11: `legacy/c_api/particle_ids.h`

```
1 #define MAX_EMITTED_PARTICLES 8
2
3 typedef struct DecayEvent {
4     int parent_species_id;
5     int daughter_species_id;
6
7     int emitted_particle_ids[MAX_EMITTED_PARTICLES];
8     size_t emitted_count;
9
10    double released_energy_eV;
11    double deposited_energy_fraction;
12    double transported_energy_fraction;
13    double local_damage_score;
14
15    double event_time_s;
16    double confidence;
17 } DecayEvent;
18
19 void decay_event_init(DecayEvent* ev);
20 int decay_event_add_emitted(DecayEvent* ev, int particle_id);
```

Listing 12: `legacy/c_api/decay_event.h`

Test Suite — `test_particle_id_decay.cpp`

The test executable runs 76 assertions across 8 groups and exits non-zero on any failure.

```

1 #define CHECK(cond, msg) do { \
2     ++tests_run; \
3     if (cond) { ++tests_passed; std::printf(" [PASS] %s\n", msg); } \
4     else      { std::printf(" [FAIL] %s (line %d)\n", msg, __LINE__ \
5         ); } \
6 } while(0)

```

Listing 13: Test harness macro and structure

```

1 auto ev = make_alpha_decay(
2     /*parent_species_id=*/ 942390,
3     /*daughter_species_id=*/922350,
4     /*released_energy_eV=*/ 5.24e6,
5     /*event_time_s=*/      1.0,
6     /*confidence=*/        0.98
7 );
8
9 CHECK(ev.parent_species_id == 942390, "parent ID");
10 CHECK(ev.daughter_species_id == 922350, "daughter ID");
11 CHECK(ev.emitted_count == 1, "one emitted particle");
12 CHECK(ev.emitted_particle_ids[0] == ParticleID::alpha_particle, "
    emitted alpha");
13 CHECK(ev.released_energy_eV > 5.0e6, "energy > 5 MeV");
14 CHECK(ev.deposited_energy_fraction > 0.8, "high deposition");
15 CHECK(ev.local_damage_score > 0.8, "high damage");

```

Listing 14: Alpha decay test — Pu-239 to U-235

```

1 DecayEvent ev {};
2 for (int i = 0; i < 8; ++i) {
3     CHECK(ev.add_emitted(ParticleID::gamma_photon), "add succeeds");
4 }
5 CHECK(!ev.add_emitted(ParticleID::gamma_photon), "9th add fails (
    overflow)");
6 CHECK(ev.emitted_count == 8, "count capped at 8");

```

Listing 15: Overflow guard — capacity capped at 8

```

1 KineticEventRecord rec {};
2 rec.kind = EventKind::decay;
3 rec.payload = make_alpha_decay(942390, 922350, 5.24e6, 1.0, 0.98);
4
5 bool dispatched = false;
6 if (auto* d = std::get_if<vsepr::physics::DecayEvent>(&rec.payload))
7 {
8     dispatched = true;
9     CHECK(d->parent_species_id == 942390, "variant: parent ID");
10    CHECK(d->emitted_count == 1, "variant: emitted count");
11 }
12 CHECK(dispatched, "variant dispatch succeeded");

```

Listing 16: Variant dispatch test

```

1 NuclearState pu239 { 942390, 94, 239 };
2
3 auto da = alpha_daughter(pu239);

```

```

4 CHECK(da.Z == 92, "alpha daughter Z = 92 (U)");
5 CHECK(da.A == 235, "alpha daughter A = 235");
6
7 auto db = beta_minus_daughter(pu239);
8 CHECK(db.Z == 95, "beta- daughter Z = 95 (Am)");
9 CHECK(db.A == 239, "beta- daughter A = 239");

```

Listing 17: Daughter-state arithmetic test

Test Results

Group	Coverage	Assertions
Constexpr queries	reserved / transportable / bookkeeping / defect	19
to_string	all 15 known IDs + unknown sentinel	16
Alpha decay	parent, daughter, emitted, energy, fractions	8
Beta-minus decay	two emitted particles, fractions	5
Gamma decay	one emitted, transport fraction, damage	5
Overflow guard	8 successful adds + 1 failure + count check	10
Variant dispatch	decay + transport variant paths	8
Daughter arithmetic	α , β^- , β^+ Z and A values	6
Total		77

Compile and run:

```

1 g++ -std=c++20 -I include -o test_particle_id_decay \
2     tests/test_particle_id_decay.cpp
3 ./test_particle_id_decay
4 # Results: 76 / 76 passed

```

Listing 18: Build and execute

Integration with Existing Infrastructure

Orthogonality with DecayMode and DecayType

Three enums address three different questions:

Enum	Location	Answers
vsepr::nuclear::DecayMode	decay_chains.hpp	How does the chain progress?
atomistic::nuclear::DecayType	nuclear_stability.hpp	What mode dominates at this Z?
vsepr::physics::ParticleID	particle_id.hpp	What was emitted?

`DecayMode::Alpha` in `decay_chains.hpp` classifies a step in the Th/U/Ac natural series. `ParticleID::alpha_p` identifies the He-4 nucleus that was emitted from that step as a bookkeeping object — they are complementary, not competing.

Relationship to ElementDescriptor

`ElementDescriptor` (in `include/core/element_descriptor.hpp`) carries the full identity fields for an element or isotope, including: `decay_clock` (half-life countdown in seconds), `metastable` (nuclear isomer flag), `mass_number` A , `neutron_number` N , and `isotope_label`.

`NuclearState` in the builder layer carries only `species_id`, `Z`, and `A` — the minimum needed for transition arithmetic. Full identity resolution defers to `ElementDescriptor`.

Layer Mapping

Layer	Files	Role
A (identity)	<code>particle_id.hpp</code> , <code>particle_ids.h</code>	Signed-int namespace
B (event)	<code>decay_event.hpp</code> , <code>decay_event_builder.hpp</code>	Physics payload
C (kinetics)	<code>kinetic_event_kind.hpp</code> , <code>kinetic_event_record.hpp</code>	Scheduler wrapper